Leginkább Habzda Fruzsina munkája

• Írd fel egy exportált rutin és egy exportált creation procedure szerződését Hoare-hármasok segítségével, és magyarázd el a különbséget!

Rutin: {INV ∧ PRE} rutin {INV ∧ POST}

Creation procedure: {PRE} creation {INV ∧ POST}

Ahol INV az osztály invariánst, PRE és POST a rutin/creation elő- és utófeltételét jelenti.

Különbség: egy creation procedure egy osztálypéldány elkészítéséért felelős, így az osztály invariáns a végrehajtás kezdetén még (általában) nem teljesül, viszont a creation procedure lefutása után (illetve bármely egyéb rutin lefutása után) igaznak kell lennie.

• Mi a szerepe a like kulcsszónak az Eiffelben?

A like egy kulcsszó, melyet változókkal párosítva használjuk, és azt jelenti, hogy olyan típusú mint az adott változó.

pl.: like Current = ugyanaz a típus, mint az aktuális objektum típusa

kapcsolt (anchored) típusoknál használatos

• Mi a mixin-inheritance lényege?

Olyan program nyelvi konstrukció, ami valamilyen értelemben helyettesíti a többszörös öröklődés előnyeit elkerülve azok problémáit (EIFFEL-ben nincs ilyen!!).
A típusparaméterből való öröklődést jelenti (Ada, C++), gyakorlatilag a többszörös öröklődés linearizálása.
Példa: (EIFFLE-es, de ILYEN NINCS BENNE!!!!)

class Numeric[T] inherit T … end
class Comparable[T] inherit T .. end
class Rational inherit Comparable[Numeric[Any]] .. end

Itt a Rational örököl a Comparable, a Numeric és az Any osztályból is, mégsem alkalmaztunk az EIFFEL-ben megszokott többszörös öröklődést. Itt természetesen

az Any helyére behelyettesítve tovább lehet fűzni az öröklődési láncot.
Nincsenek konfliktusok, nem lesz “diamond” öröklődés.

• Milyen szerződések vonatkoznak az Eiffel rutinokra? Hogyan befolyásolja ezt az öröklődés? Hogyan a felüldefiniálás? Magyarázd el a kapcsolódó fogalmakat és szabályokat. Fogalmazd meg a szerződéseket Hoare-hármasok segítségével! (7. ea vége)

A rutinokra a rutin szerződések és az osztály szerződések vonatkoznak. (rutin require, ensure, osztály invariant)

Az invariant klózba beletartozik minden megörökölt invariáns is. (ÉS)

(Az invariant klóz alatt megadott feltételeknek az osztály creation procedure lefutása után fenn kell álljanak, valamint minden művelet előtt és után is igaznak kell legyenek.)

Öröklés során a rutin előfeltétel gyengíthető (VAGY), az utófeltétel szigorítható (ÉS).

Felüldefiniálás során egy rutin szerződését úgy lehet módosítani, hogy az a kliensek számára ne “rontsa el” a szerződést. A szerződés módosításhoz használt kulcsszavak: require else, ensure then.

Ha r' felüldefiniálja a megörökölt r rutint;

PRE(r') = PRE(r) ∨ require_else r'
POST(r') = POST(r) ∧ ensure_then r'

Így a szerződés:
{ PRE(r') ∧ INV(C) } $C.r'$ { POST(r') ∧ INV(C) }

inicializációkor:
{ PRE(r') } $C.r'$ { POST(r') ∧ INV(C) }

(lf-fel: $Q \Rightarrow Q' \Rightarrow$ lf(r' , R') $\Rightarrow$ lf(r' , R))

Amennyiben hiányzik valamelyik szerződés:

Ha nincs require → require True
Ha nincs require else → require else False
Ha nincs ensure → ensure True
Ha nincs ensure then → ensure then True

• Milyen viszonyban kell álljanak az Ai és a Bj típusok ahhoz, hogy fennálljon az alábbi altípus reláció (matematikailag helyesen)? Indokold válaszod! (A1 -> A2) -> A3 -> A4 <: (B1 ->B2) -> B3 -> B4

(A' → B') <: (A → B) akkor teljesül, hogy ha A <: A' és B' <: B.

(A1 → A2) → A3 → A4 <: (B1 → B2) → B3 → B4 másképp:
(A1 → A2) → (A3 → A4) <: (B1 → B2) → (B3 → B4)
A fenti akkor teljesül, hogy ha (B1 → B2) <: (A1 → A2) és (A3 → A4) <: (B3 → B4) teljesülnek.
(B1 → B2) <: (A1 → A2) akkor teljesül, ha A1 <: B1 és B2 <: A2 teljesülnek
(A3 → A4) <: (B3 → B4) akkor teljesül, ha B3 <: A3 és A4 <: B4 teljesülnek
Tehát A1 <: B1, B2 <: A2, B3 <: A3 és A4 <: B4 altípus relációknak kell fennállniuk.

• A strukturális altípusosságot hogyan szokás definiálni direktszorzat (tuple) típusokra?

Kovariancia jellemző rájuk, a Tuple nem tisztán funkcionális stílusban van használva, de olyan, mint a funkcionális nyelvekben lehetne:
TUPLE[INTEGER,STRING] <: TUPLE[INTEGER,ANY] <: TUPLE[INTEGER] <: TUPLE[ANY] <: TUPLE

A kovariancia miatt TUPLE[INTEGER,STRING] <: TUPLE[INTEGER,ANY] igaz, mert integer altípusa integer-nek és string is altipusa any-nek.

TUPLE[INTEGER,ANY] <: TUPLE[INTEGER] igaz, mert rendezett n-es esetében az altípusban több komponens is lehet, az egyetlen kikötés hogy az első típus feleljen meg az altípus első paraméterének (itt: integer-integer)

Formálisan: $n \ge m \land \forall i\in[1..m]: A_i <: B_i \implies \times^n_{i=1} A_i <: \times^m_{i=1} B_i$

$$n \geq m \land \forall i \in [1..m] : A_i <: B_i \implies \times_{i=1}^{n} A_i <: \times_{i=1}^{m} B_i$$

• A strukturális altípusosságot hogyan szokás definiálni rekord típusokra?

Lásd tuple kérdés. Különbség: nevesítve vannak az adattagok, nem pedig az indexük azonosítja őket.

Formálisan: B rekord típus minden szelektora szerepel az rekord típusban is, és minden ilyen szelektorra az A-beli típus altípusa a B-beli típusnak, akkor $A <: B$

• Mire való az old kulcsszó az Eiffelben? Magyarázd el, fogalmazd meg a rá vonatkozó szabályokat, illusztráld használatát példán!

Az old kulcsszó a rutin utófeltételben (ensures) szokott szerepelni. Azt jelzi, hogy a rutin megkezdésekori értéke egy változónak. Ilyen esetben az Eiffel külön lementi magának a változó ezen értékét, hogy majd a feature végén tudja használni. A separation logic megadásakor, a frame condition írásában használatos (mi változott, mi maradt változatlan).

példa:

```
deposit(amount:INTEGER)

require
 positive_amount: amount>0

do
 balance := balance + amount

ensure
 deposit_valid : balance = old balance + amount

end
```

• Hogyan működik, miért és mire jó az átnevezés az öröklődés során? (7. ea)

Különböző bázistípusokban lévő műveleteket átnevezhetünk, hogy a névütközést elkerüljük. (lásd név, átnevezés)

• Mit jelent a mixin inheritence? Mi az előnye? Minek az alternatívája?

Olyan program nyelvi konstrukció, ami valamilyen értelemben helyettesíti a többszörös öröklődés előnyeit elkerülve azok problémáit (EIFFEL-ben nincs ilyen!!). A típusparaméterből való öröklődést jelenti (Ada, C++), gyakorlatilag a többszörös öröklődés linearizálása.
Példa: (EIFFLE-es, de ILYEN NINCS BENNE!!!!)

class Numeric[T] inherit T … end
class Comparable[T] inherit T .. end
class Rational inherit Comparable[Numeric[Any]] .. end

Itt a Rational örököl a Comparable, a Numeric és az Any osztályból is, mégsem alkalmaztunk az EIFFEL-ben megszokott többszörös öröklődést. Itt természetesen az Any helyére behelyettesítve tovább lehet fűzni az öröklődési láncot. Nincsenek konfliktusok, nem lesz “diamond” öröklődés.

• Milyen szerződésnek kell megfelelnie a retry kulcsszóval végződő kivételkezelő ágaknak? Fogalmazd meg Hoare-hármas segítségével!

Ez újrapróbálkozást jelent, ahol a retry hatására a rutin törzs újraindul. Ebben az esetben a rutin előfeltételt biztosítanunk kell, ezt a reset művelettel tehetjük meg ()itt szükséges megadni, hogy mit állítanánk vissza). (ez azonban nincs benne az ECMA szabályban)

{ True } rescue_with_retry r { Inv$C$ $\wedge$ pre r }

C osztály, r rutin

• Milyen szerződésnek kell megfelelnie egy ciklusnak? Fogalmazd meg Hoare-hármasok segítségével!

A ciklus szerződése az invariant és variant klózban található.

from INIT
 invariant INV
 variant VAR
 until COND
 loop BODY
 end

(INIT, BODY összetett utasítások, INV, COND: $A \rightarrow$ BOOLEAN)

$Q \Rightarrow lf$(while π do B, R) //ahol Q az előfeltétel, R az utófeltétel
 $\{Q\}$ while π do B $\{R\}$

Egy ciklus mikor felel meg a szerződésének?

1. $\{true\}$ INIT {INV} //előfeltételből következik az invariáns
 2. –
 3. {INV $\wedge$ ¬COND} BODY {INV} // inv+!ciklusfeltétel -> terminálás

4. INV ⇒ VAR ≥ 0 //terminál fgv (ciklus iterációk)
5. $\forall v$: {INV $\wedge$ ¬COND $\wedge$ VAR = v} BODY {VAR ≤ v − 1)} //term. fgv értékének csökkenése

• Mire való az “old”? Mikor, hogyan, mire használjuk? Mutass példát is!

lásd előző old-os kérdés

• Mit szokás bináris műveletnek nevezni az objektumorientált programozás-ban?

Két (azonos típusú) paraméterrel rendelkező művelet. Például az aritmetikai műveletek. (+,-,*,...)

pl.:

```
class FRACTION
...
feature
 numerator, denominator: INTEGER
 ...
 plus alias "+" ( other: like Current ): like Current
  ...
end
```

• Mi a különbség az =, a ~, az is_equal, az equals, a standard_is_equal és az is_deep_equal között? Mi közük van egymáshoz?

Referencia típusoknál:

- referenciák egyenlősége (object identity) a = b
- tartalmi egyenlőség (object equality)
  van sekély, mély és egyedi (a ~ b)
  sekély: (standard_equal) az objektum mezői páronként =-vel összehasonlítva
  mély: (deep_equal) az objektum mezői páronként mélyen egyenlőek, rekurzívan
  egyedi: (is_equal, a ~ b) ha az egyedi egyenlőségvizsgálat nincs felüldefiniálva, megegyezik a sekéllyel
- **=** ⊂ **standard_equal** ⊆ **equal** ≡ **~**
  = ⊂ **standard_equal** ⊆ **deep_equal**

Kifejtett (expanded) típusoknál:

- a = b és a ~ b jelenti a tartalmi egyenlőséget (itt nincs értelme object identity-nek)
- sekély (standard_equal), mély (deep_equal), egyedi (is_equal) egyenlőség vizsgálatok
- **standard_equal** ⊆ **=** ≡ **equal** ≡ **~**
  standard_equal ⊆ **deep_equal**

• Hogyan szabályozza az Eiffel a láthatóságot? Miben különbözik ez a más nyelvekben megszokottól? (7. ea)

Az Eiffel szelektív láthatóságot alkalmaz. A láthatóság nem osztályokra, hanem objektumokra vonatkozik más nyelvekkel ellentétben. Tehát, ha priváttá tesszük egy adattagját, azt az azonos osztályból származó objektumok nem érhetik el.

(kiegészítésre szorul)

Az adott feature klóz után kapcsos zárójelben adhatjuk meg az osztályt.
Alapvetően az osztály és annak leszármazottai érhetik el az adott featureöket.
A feature {MYCLASS} a MYCLASS és leszármazottai számára elérhető.

feature -- mindenki számára

feature {ANY} -- az ANY leszármazottainak

feature {NONE} -- a NONE leszármazottainak (senkinek)

feature {} -- régi szintaxis (senkinek)

(a láthatóság kontravariáns)

• Milyen szerződés vonatkozik egy retry nélküli kivételkezelőre? Írd le Hoare-hármassal!

Ezt organized panic-nak nevezik, ha retry nélkül fejeződik be a rescue. Ez azonban azt jelenti, hogy az utófeltételt nem tudtuk elérni, és mivel sikertelen volt a művelet, az osztályvariánst vissza kell állítani a művelet előtti állapotára.

{True} rescue_without_retry r {Inv$C$}

C osztály, r rutinja

• Milyen szerződés köti az alosztályban felüldefiniált műveletet? Írd le Hoare-hármassal!

lásd a “Milyen szerződések vonatkoznak az Eiffel rutinokra? Hogyan befolyásolja ezt az öröklődés? Hogyan a felüldefiniálás? Magyarázd el a kapcsolódó fogalmakat és szabályokat. Fogalmazd meg a szerződéseket Hoare-hármasok segítségével!” kérdésnél

• Mi a family polimorfizmus alapproblémája?

Absztrakt implementációk esetén, ha konkrét megvalósításhoz akarok eljutni, akkor szándékolatlan kapcsolatok jönnek (jöhetnek) létre az altípusosság miatt
Kivédés: -kódismétlés (elhagyni az öröklődést) - JAVA
-öröklődés osztálycsaládok között (nem csak osztályok között) - Scala

• Milyen esetekben áll fenn altípus reláció paraméterezett típusok között az Eiffelben? Mit jelent itt a frozen?

Eiffel-ben általában a típusparaméter is kovariáns:
-ha class STACK[T] …., akkor STACK[INTEGER] <: STACK[ANY]
Ha így használjuk a generikus típusokat az elrontja a statikus típusrendszert
Kivéve:
Ha használjuk a frozen kulcsszót, ekkor a típusparaméter invariáns és nem kovariáns:
-ha class STACK[frozen T] …, akkor STACK[INTEGER] <: STACK[ANY] nem áll fenn

Összességében ha mindig frozen típusparamétereket használuk EIFFEL-ben, azzal nem igazán lehet mellélőni (így ez az egész hasonlít a JAVA generikus paraméter szabályaihoz)

(-------) -> sry ezt átírom, de nem törlöm ki mert most nem erre gondol

frozen: befagyasztás, nem definiálható felül (Javaban final)

lehet class is frozen, ebből nem származtathatunk

biztonság kritikus műveleteknél használatos, és a fordító több optimalizációt tud ezeken végrehajtani

(amire a frozen vonatkozik, az nem lehet deferred = absztrakt)

• Mit jelent az attached (az Eiffel szabványban „!”) és a detachable (a szabványban „?”)?

Az attached (régi szabványban „!”) egy kulcsszó, azt jelent, hogy a változó nem vehet fel VOID értéket (tehát nem lehet nullpointer). Régen az attached volt kötelezően kiirandó, most azonban már ez a default. (Az expanded class csak attached lehet)

A detached (régi szabványban „?”), szintén egy kulcsszó, azt jelenti, hogy a változó felveheti a VOID értéket. Régen ez volt a default, és nem kellett kiírni, de ma már ez a kötelezően kiirandó.

pl.: name: attached STRING

Amúgy ezek a Cecil nyelv jelölésrendszeréből lettek átvéve.

érdemes még megemlíteni, hogy az attached XY osztály valódi altípusa a detached XY osztály-nak
pl.: attached ACCOUNT <: detachable ACCOUNT

és hogy VOID-ság vizsgálata helyett az attached-séget tudjuk vizsgálni Eiffelben (dinamikus típusellenőrzés)

• Milyen módon kezeli a kivételeket egy rutin, ha nincs benne rescue?

Az Eiffelben előre tervezünk, nem utólag kezelünk, így inkább felelőségkezelés a van kivételkezelés helyett

De előfordulnak kivételes esetek, ekkor a rutinba egy rescue utasítássorozatot szokás írni.

Ha ez elmarad, viszont egy kivétel keletkezik a rutin törzsének végrehajtása során (mely kibillenti a programot a normál végrehajtásból), lekezelés hiányában továbbterjed a hívási lánc mentén, mely a program leállását eredményezheti.

Szerződésszegés

Stack trace

(különben minden osztály, minden rutinjába kerül egy rescue, az ANY base osztályból. A fordító generálja beléjük a default_rescue-t)

• Mi a default_create?

Ha egy osztály nem ad meg creation procedure-t, akkor benne az ANY osztályból megörökölt azon művelet lesz az egyetlen creation procedure, amelyet az ANY osztályban default_create-nek neveznek. (Megörökli, mivel az összes osztály az ANY-ből származik) Gyakorlatilag egy üres alapértelmezett konstruktor.

Ha egy osztályban az ANY-ből megörökölt default_create az (egyik) creation procedure, akkor annak meghívása elhagyható. Ez akkor is működik így, ha átnevezzük helyben a default_createt
pl.: create {POINT} //nem kell mögé írni a .default_create-et

• Hogyan szól a Liskov-féle helyettesítési elv (substitution principle)?

Az Barbara Liskov-féle helyettesítési elv (LSP) az objektum-orientált programozás öt alapelvének egyike, amelyeket SOLID-elvekként ismerünk. Hangsúlyozza az alaposztály viselkedés megőrzésének fontosságát, amikor kiterjesztik vagy felülírják egy származtatott osztályban.

Liskov helyettesítési elve kimondja:

"Az altípusoknak helyettesíthetőnek kell lenniük az alaptípusukkal anélkül, hogy megváltoztatnák a program helyességét."

pl: ha egy S osztály a T osztály altípusa, a T osztály példányai lecserélhetők az S osztály példányaira anélkül, hogy ez befolyásolná a program helyességét.

Ez azt jelenti, hogy egy származtatott osztálynak ki kell terjesztenie az alaposztály viselkedését anélkül, hogy magának az alaposztálynak a viselkedése megváltozna.

• Mi a kapcsolt (anchored) típus az Eiffelben? Mutass példát arra, hogy miért jó!

nem kell újradefiniálni, adaptív típus, like kulcsszó használata
A típus definíciójában nem a konkrét típust adjuk meg, hanem azt mondjuk, hogy az a típus mint az aktuális objektum:

like Current: ugyanaz a típus, mint az aktuális objektum típusa

pl.:

class HOLDER[T]

create
 set

```
feature
 item: attached T assign
 set set( value: like item )
  do item := value
  ensure item = value
  end

end
```

Előnye például, hogy a 32 kovariánst visszatérési típus megvalósításához megspóroljuk az újradefiniálást (twin feature), adaptív a típus.

• Milyen szerződésnek kell eleget tegyen egy creation procedure? Írd le Hoare-hármas segítségével!

▶ Ha a létrehozó műveletek ($C.cp$) beállítják az invariánst

▶ Ha a műveletek ($C.r$) megőrzik az invariánst

{REQcp} cp {ENScp ∧ INVC}

{REQr ∧ INVC} r {ENSr ∧ INVC}

• Mire való az only az Eiffelben (illetve a strip a jelenlegi implementációkban)?

Az only, illetve strip kulcsszó a rutin utófeltételben (ensures) szokott szerepelni. A separation logic megadásakor, a frame condition írásában használatos (mi változott, mi maradt változatlan).

Az only azt jelenti, hogy csak az az adott változó változott, a többi pedig nem.

A strip-nél több ilyen azonosító is megadható, és azt jelenti, hogy a felsoroltak-at figyelmen kívül hagyjuk, tehát;

strip (balance) ~ old strip (balance)

azt jelenti, hogy csak a balance értéke változott a rutin végrehajtásakor, az objektum többi attribútuma pedig változatlan maradt.

• Mi a különbség a strukturális és a nominális altípusosság között?

Nominális altípusosság, amikor az altípus relációt a programozó expliciten megadja, például (az Eiffelben az inherits).

A strukturális altípusosság, amikor két objektum definíciójából, szerkezetéből következik az altípus reláció (azonos adattagok, egyik ugyanaz mint a másik csak bővebb). Típusok felépítése szerinti induktív definíció.

Strukturális altípusosság esetén `C <: P` akkor áll fenn, ha `C`-ben minden rutin, attribútum, stb. megtalálható, amely `P`-ben is (azaz ha szerkezetileg `P` részhalmaza `C`-nek).

Ezzel elletétben nominális altípusosság esetén csak akkor áll fenn az altípus reláció (`C <: P`), amennyiben a `C` osztály expliciten a `P` osztály altípusának van megjelölve. Ilyen például az Eiffel nyelv: az `inherits` kulcsszóval lehet altípusosságot bevezetni.

Azaz bár strukturális altípusosság esetében nem szükséges, hogy a két típusnak köze legyen egymáshoz (elég, ha az egyik tartalma részhalmaza a másik tartalmának), nominális altípusosság esetében az altípus reláció csak explicit módon tud létrejönni.

• Mit jelent a polymorphic CAT-call? (9 ea)

CAT: Changed Availability or Type

Egy polimorf változón keresztül akarunk meghívni egy olyan műveletet, ami CAT tulajdonsággal bír.

Ha leszármaztatásnál

- örökölt feature-nek csökken a láthatósága (changed availability) vagy
- egy mezőnek vagy rutin paraméterének szűkül a típusa, azaz kovariáns módon változtatom a paramétert (changed type)

Ha leszármaztatásnál örökölt featurenek csökken a láthatósága, vagy mező vagy rutin paraméternek szűkítem a típusát. Paraméter szűkítés például, ha egy altípusú objektumot bázis típusú referencián keresztül érünk el, de az altípusban egy paraméterének a típusát kovariáns módot módosítottuk, viszont a referencián

keresztül az eredeti típus szerinti paramétert adjuk át neki, ekkor futás időben derül ki, hogy nem megfelelő típust kap az objektum.

Példa: CAT <: ANIMAL, MILK <: FOOD, ANIMAL.EAT(FOOT), CAT.EAT(MILK)

```
AnimalVariable = Cat
AnimalVariable.Eat(Food) -- futás idejű hiba
```

• Mit értünk az alatt, ha egy objektumművelet esetén a paraméterek non-variánsak? Milyen problémát vet fel?

Az objektumműveletek esetében a formális paramétereknek az értékét nem lehet megváltoztatni, csak használni. (Így utófeltételbe sincs értelme beleírni őket)

Ha megpróbáljuk megváltoztatni az értékét, fordítási hibát kapunk.

Ennek kikerülésére, lokális változókba kimásolható az értékük.

A nem látható első paraméter (maga az objektumunk) azonban változtatható.

Másik válasz:

A paraméterek nonvarianciája azt jelenti, hogy (elsősorban leszármazáskor) a paraméterek típusa nem változtatható meg az alosztályban.

Az Eiffel programozási nyelv nem ilyen: ott kovariánsak a paraméterek.

Invariáns paraméterek esetén egyes műveletek megvalósítása nehezebb, mintha a kovariancia engedélyezve lenne. Viszont a kovariáns paraméterek problémákat okozhatnak, hiszen megsértik a Liskov helyettesítési elvet, lásd CAT-call. Tehát a Liskov helyettesítési elv szerint a kontravariáns paraméterek lennének a helyesek, de az invariáns paraméterek is biztonságosak, azonban egyes esetekben hasznos a paraméterek esetén is a kovariancia.

Például egy `is_equal` metódus, amely eldönti, hogy a pillanatnyi osztálypéldány egyezik-e a paraméterként kapott példánnyal, hasznos, ha olyan típusú paramétert fogad, mint amilyen a pillanatnyi osztálypéldány.

Hasonlóan, néha az alosztályokban csak speciálisabb értékeket szeretnénk paraméterként elfogadni, hiszen maguk az alosztályok is az általános osztály, a bázisosztály speciális változatai. Például invariáns paraméterek esetén nem tudjuk azt leírni, hogy az `ANIMAL` osztály `eat` metódusa `FOOD` típusú értéket kaphat, de a `CAT <: ANIMAL` osztály `eat` metódusa csak `MILK` paramétert fogad el.

• Hogyan, milyen módokon örökölheti meg az A osztály a B osztály egy f feature-ét, ha C és D a B gyermeke, valamint A a C és a D gyermeke? Milyen eseteket különböztethetünk meg?

Ha nincs azonos nevű feature C-ben és D-ben, akkor gond nélkül megörökli az összes szülőosztály feature-t az A. Amennyiben C-ben és D-ben van azonos nevű feature, akkor több megoldásunk is van az ütközés elkerülésére:
1. Az egyikből (pl D) átnevezzük a feature-t öröklés során:
inherit D feature_eredeti_neve as feature_uj_neve end
2. Megtehetjük azt is, hogy az egyiket átnevezzük mint az előző esetben, de felül is definiáljuk. (redifine feature_uj_neve) (Dinamikus kötés)
3. Másik eset, ha ugyanaz a feature és ugyanazok is az implementációk (pl B-ből örökölte C is és D is, illetve nem definiálták felül), akkor A is megörökli ugyanazt a feature-t azzal az implementációval hogy B-ben is szerepel.
4. Előző esetben, ha mondjuk C felüldefiniálta az említett feature-t akkor A nem tudja megörökölni mindkét implementációt. Ilyenkor a hiba elkerülése végett, az egyik implementációt töröljük az “undefine” kulcsszóval.

Rövid válasz:

Ha ugyanazon a néven örököljük több irányból ugyanazt, csak egyet kapunk belőle (“join”)
Ha különböző néven, akkor többet
Ha több implementációját örököljük, akkor nem lehet join-olni
De öröklődésnél törölhetjük az implementációt: undefine
Különböző feature-öket nem lehet join-olni

> Ezt a problémát diamond inheritance-nek is szokás nevezni, az öröklődési gráf lerajzolt alakja miatt.

Az A osztály a B osztályt két irányból is (C-n és D-n keresztül is) megörökli. Attól függően különböztetünk eseteket, hogy C és D az f feature átnevezte-e, felüldefiniálta-e.

- f át lett nevezve
  - C-ben és D-ben eltérő nevekre: A-ban az f feature kétszer, két különböző néven fog szerepelni. A `select` kulcsszó használata kötelező, hogy dinamikus kötés esetén el tudja dönteni a futási környezet, hogy a két implementáció (C és D beli) közül melyik fusson le, amennyiben az f feature egy D statikus típusú referencián keresztül van meghívva
  - C-ben és D-ben azonos névre: olyan, mintha nem lett volna átnevezve
- f nem lett átnevezve
  - sem C, sem D nem definiálta felül az f rutint
    - Ilyenkor automatikus join történik: csak egy f rutin lesz A-ban, ami azonos módon működik, mint D-ben
  - C vagy D felüldefiniálta az f rutint
    - Ilyenkor nem lehetséges az automatikus join, hiszen nem tudja a fordítóprogram, hogy két implementáció közül melyik tartsa meg.
    - Egyik lehetőség: eldobjuk az egyik implementációt (`undefine`), ilyenkor a másik marad meg
    - Másik lehetőség: átnevezzük az egyiket, így két feature-ünk lesz, nem fognak ütközni egymással

• Milyen lehetőségeket biztosít egy kovariáns sablonparaméter?

Eifellben van lehetőségünk kovariáns típusparaméterek deklarálására (pl. `STACK[T]`). Opcionálisan, a `frozen` kulcsszó segítségével meg lehet jelölni egy típusparamétert invariánsként (pl. `STACK[frozen T]`).

A kovariáns típusparaméter a statikus típusrendszert bár el tudja rontani, de bizonyos esetekben (amikor pl. funkcionális módon használjuk a generikus osztályt, azaz csak kiolvasunk belőle adatokat és nem módosítjuk azt) hasznos tud lenni.

A kovariáns sablonparaméternek köszönhetően egy `STACK[T]` osztály esetén `STACK[INTEGER] <: STACK[ANY]`-nek, hiszen `INTEGER <: ANY`.

Több típusparaméterrel is működik a kovariancia, illetve amennyiben `DEQUEUE <: STACK`, akkor `DEQUEUE[INTEGER] <: STACK[ANY]`, stb.

`frozen` típusparaméter esetén (pl. `STACK[frozen T]`) invariancia áll fenn a típusparaméterre nézve: `STACK[INTEGER]` és `STACK[ANY]` között nincs altípus reláció.

Használat példa: legyen `LIST[T]` egy kovariáns típusparaméterrel rendelkező osztály és legyen egy szekvenciát feldolgozó osztály, amely `LIST[ANY]` típusú paraméterrel rendelkezik. Ennek az osztálynak átadhatunk egy `LIST[INTEGER]` példányt, hogy azzal dolgozzunk: nem kell explicit egy `LIST[ANY]` példányt létrehoznunk.

• Mi a név szerepe, és miért fontos az átnevezés az Eiffelben? (7. ea)

A név egy dolog azonosítására szolgál. (Dolog itt; művelet, adattag, változó, stb)

Egy kontextuson belül egyértelműnek kell lennie a névnek, de kontextusok között az adott (ugyanaz a) dolog neve változhat.

Névütközések elkerülése miatt lényeges, hogy át lehet nevezni a dolgokat

.

• Mi a különbség a nyilvános és a privát öröklődés között?

Nyilvános öröklődés esetén az alosztály megörökli az ősosztály összes feature-ét (azok láthatóságát is) és az ősosztály alosztályává válik. Privát öröklés esetén az

osztály (direkt nem alosztálynak nevezem) ugyanúgy megörökli az öröklött osztály (direkt nem ősosztály) összes feature-ét, viszont NEM válik annak alosztályává, tehát ez csak kódöröklést jelent. Fontos megemlíteni, hogy a feature-ök neve, láthatósága, felüldefiniálása ugyanúgy működik mind2 esetben.

• Milyen problémákat vet fel a többszörös öröklődés?

Az Eiffel támogatja a több szülőosztályt (ez problémákat vethet fel pl: diamond, névütközés). Gazdagabb lehetőségeket kínál mint a C+, tud publikus és privát öröklést is.
Többszörös öröklődés esetén MAXIMUM egy konform, a többi privát.
Probléma:

-Névütközés:

minden feature neve különböző kell legyen egy adott osztályban, amennyiben egy nevet több helyről örököl az oszutály, akkor ez hibát okoz.
Megoldása: lehetőség van átnevezni az öröklött osztályok egyes(vagy mind) feature-jét: “rename is_right as holds end” ekkor a létrehozott osztály kontextusában az is_right függvény holds névvel érhető el, a dinamikus kötés pedig feloldja majd a névváltást.

-Ugyanazt a feature-t több irányból különböző implementációval:

azonos névhez tartozó feature-ök NEM térhetnek el implementációjukban, vagy fordítási hibát kapunk megoldás: (lsd fent) + eldobás is lehetséges: undefine is_equal end

-Diamond:

-Ha ugyanazon a néven örököljük több irányból ugyanazt, csak egyet kapunk belőle (“join”)

-Ha különböző néven, akkor többet

-Ha több implementációját örököljük, akkor nem lehet join-olni

Kivéve: ha töröljük

-Különböző feature-öket nem lehet join-olni

• Mik az expandált típusok?

expanded: kifejtett osztály (pl INTEGER)

Ez nem a heap-en fog létrejönni, hanem helyben (általában a memóriában, stacken). Tehát az egész osztály, az attribútumaival együtt itt jön létre, és itt él. Nem referencián keresztül, hanem közvetlenül elérhetőek.

Viszont ha egy expanded class-t tartalmaz egy nem expanded class, akkor a sima class objektuma létrejön a heapen (dinamikus memóriában), és azon belül jön létre az expanded class.

(Az expanded class csak attached lehet, polimorfizmus nem használható rájuk, se aliasing, de hatékonyság szempontjából jók)

• Hogyan számoljuk ki egy osztály invariánsát? (. . . öröklődés . . .)

Az invariant klóz alatt megadott feltételeknek az osztály creation procedure lefutása után fenn kell álljanak, valamint minden művelet előtt és után is igaznak kell legyenek.

Az osztály invariánsa és az összes szülő invariánsa ÉS-el összekapcsolva értendő.

• Mit értünk sekély, illetve mély másoláson?

Sekély másolás: Az eredeti és a másolat sekély egyenlőségvizsgálat szerint egyenlőek. (standard_copy)

Mély másolás: Az eredeti és a másolat mély egyenlőségvizsgálat szerint egyenlőek. (deep_copy)

• Milyen lehetőségek vannak a referenciák nem-ürességének leírására az Eiffelben?

már az attacheden kívül?... -> TODO: a check-re gondol

• Mi az F-bounded polimorfizmus? Mutass rá példát Javában!

A típusparaméterre adott korlát, saját magától függ. (EIFFEL-ben nincs ilyen)

public static <T extends Comparable<T>>
T max (List<T> list);

Ez annyit jelent, hogy a T-re szeretnék deffiniálni egy maximum kiválasztást, aminek elengedhetetlen része az, hogy a T összehasonlítható legyen magával. -> a T-re adott korlátozás függ a T-től (T extends Comparable<T>).

• Mit jelent a “frame condition” kifejezés? Hogyan írunk ilyet az Eiffelben?

A keretfeltételek azt írják le, hogy mely modellelemek változhatnak az egyik rendszerállapotból a másikba való átmenet során, és így elkerülhető a nem kívánt viselkedés.

Az old kulcsszóval elérhető egy adatunk paramétertérbeli értéke, így utófeltételben tudunk vizsgálatot írni az aktuális és a kezdeti érték egyenlőségére (vagy egyenlőtlenségére), ezzel kifejezve a változást / nemváltozást: ensure frame: adat = oldadat.

lásd old, only, strip …

• Milyen kifejezések szerepelhetnek értékadás baloldalán az Eiffelben?

?

Objektum a saját mezőjét írhatja, objektumon belül saját adattag szerepelhet az értékadás bal oldalán.

Lokális változót lehet módosítani az adott műveleten belül, szerepelhet az értékadás bal oldalán.

Formális paraméter nem írható, nem szerepelhet az értékadás bal oldalán (segédváltozó bevezetése szükséges).

Result kulcsszó

• Mire jó a debug utasítás az Eiffelben?

A szoftverfejlesztés támogatására került a nyelvbe.

pl.: debug(Debug_key) … end

Az eközötti utasítások csak akkor hajtódnak végre, ha a debug be van kapcsolva és az indításkor az adott Debug_key meg van adva. Ekkor a fordító, a megadott Debug_key-el rendelkező utasításokat belefordítja a kódba.

• Hogyan történik a paraméterátadás az Eiffelben? Térj ki az expandált típusokra is! (5. ea)

A paraméter típusa határozza meg, nem az átadott objektum típusa.

Kifejtett tipusúnál érték szerinti (call-by-value) paraméterátadás van.
pl.: denominator: INTEGER attribute Result := 1 end

Referencia tipusúnál megosztás szerinti (call-by-sharing) van.

Probléma akkor adódhat, ha saját magát adjuk át formális paraméterként is:

Csinálunk egy FRACTION törtszám osztályt, és definiáljuk a divide_by(other: attached FRACTION) eljárását. Ekkor ha f.divide_by(f) eljáráshívás történik, akkor a numerator := numerator * other.denominator értékadás elvégzésénél már megváltoztatjuk az other numenatorját is, így a denominatort majd rosszul számoljuk ki, hiszen a Current és az other ugyanazokra az objektumokra hivatkoznak.

• Milyen elvárások fogalmazhatók meg egy egyenlőségvizsgálat műveletre?

Az egyenlőségvizsgálat műveletnek egy ekvivalenciarelációt kell definiálnia: reflexív, tranzitív, szimmetrikus.

Továbbá konzisztenciát is elvárunk az egyenlőségvizsgálattól: ha nem változtatjuk sem `x`-et, sem `y`-t, akkor az `x ~ y` művelet eredményének sem kéne változnia.

Az altípusosság egy részbenrendezés, így az egyenlőségvizsgálattal sokszor nehezen hozható összhangba.

Monomorf megközelítés: egy osztálypéldány csak olyan példánnyal tekinthető egyenlőnek, amelyik pontosan ugyan ahhoz az osztályhoz tartozik. Az adott osztály altípusának egy példánya nem tekinthető egyenlőnek a bázistípus egy példányával. Ez a megközelítés általában adathordozó objektumok esetén hasznos, a szimmetria így triviálisan teljesíthető. Ilyenkor általában nem is kívánatos az altípus reláció megjelenése és hasznosabb lehet a privát öröklődés vagy a kompozíció használata öröklődés helyett.

Polimorf megközelítés: egy bázisosztályban megadhatjuk az egyenlőségvizsgálat működését, amelyet aztán az alosztályok nem változtatnak meg (maximum hatékonyabbá tesznek egyes speciális esetekben). Ez olyankor hasznos, amikor a bázisosztály valamilyen általános koncepciót ábrázol, aminek több lehetséges megvalósítása lehet: például lehet a `List` a bázisosztály és tekinthetünk egy `LinkedList` és egy `ArrayList` példányt egyenlőnek, ha azonos indexen egyenlőnek tekinthető elemek helyezkednek el.

• Mi a különbség egy exportált és egy nem exportált feature szerződésében? Mutasd be Hoare-hármas segítségével!

• Mit értünk multimethodon?

Bizonyos objektumorientált nyelvekben a dinamikus kötés nem egyetlen kitüntetett paraméter dinamikus típusán működik, hanem az összes paraméter dinamikus típusát figyelembe veszi. Például az Eiffelben az a.b(c) rutinhívásnál az a és a c változók statikus típusa alapján dől el, hogy a b rutin hívása értelmes-e, és az a változó dinamikus típusa alapján dől el, hogy a művelet mely osztályban definiált implementációja hajtódik végre. Egy multimethodokat (azaz multiple dispatchet) támogató nyelvben az a és a c dinamikus típusa alapján választódik ki a végrehajtandó kód (azaz az a statikus típusában, illetve leszármazottaiban több olyan b művelet is lehet, amelynek a paramétere a c statikus típusának valamilyen altípusába tartozik).

A multimethodokat támogató nyelvekben az úgynevezett bináris műveletek (azaz amikor a művelet paraméterének típusa a fogadó objektum típusával kell megegyezzen, mint mondjuk egyenlőségvizsgálatnál) megvalósítása megoldott, hiszen minden osztályban megírhatjuk az arra az osztályra jellemző (és a fogadó és paraméter objektumot ugyanazzal típusozó) implementációt, és a dispatch során a legjobban illeszkedő implementáció hívódik meg. A szimmetriáról a multiple dispatch gondoskodik.

• Mik a virtuális típusok? Mire jók?

SCALA-ban pl van ilyen. Bizonyos esetekben kiválthatja a generikus paramétereket.
Az osztályomban lesz egy olyan tagtípus, amelyet utána az alosztályban lehet specializálni (hasonlóan mint a fgv-nél, de itt a típus specializálható).
A konkrét leszármazottban meg KELL adni a deffinicióját a típusnak
Általánosan elmondható hogy statikus típusosság szempontjából nem szerencsés, ugyan úgy mint a kovariáns paraméter (anchored type in EIFFEL)
Scala: itt részleges virtuális típusok vannak (statikusan helyes típus rendszert eredményez)

Scala példa: (itt a type L type R a 2 virtuális típus) -> a második class definícióból ofc

```
class Pair[L,R]( left: L, right: R )
val v = new Pair[Int,String](1,"hi")
v.right

abstract class Pair {
    type L
    type R
    val left: L
    val right: R
}
val v = new Pair{ type L=Int; type R=String;
                  val left=1; val right="hi" }
v.right
```

• Mit nevezünk parametrikus polimorfizmusnak?

• Mit nevezünk altípusos polimorfizmusnak?

• Mit jelent a bounded parametric polymorphism? Milyen megszorításokat lehet tenni az Eiffelben?

Bounded parametric polymorphism : bizonyos feltételeket kielégítő típusokkal lehet paraméterezni egy polimorf osztályt. Ez lehet felső korlát (upper bound), ami egy adott osztály leszármazottaira korlátoz, vagy lehet alsó korlát (lower bound) (pl. Javában).

Eiffel megszorítások:
class VECTOR [...] … esetén

VECTOR[G -> ADDABLE] -- ADDABLE features usable on G
HASHTABLE[K -> HASHABLE, V] -- more type parameters
VECTOR[G -> {ADDABLE, HASHABLE}] -- more constraints
VECTOR[G -> ADDABLE create make end] -- instantiation ok
VECTOR[?G -> ADDABLE] -- self-initializing type required
VECTOR[G -> ADDABLE rename add as plus end]
-- provide a new feature name to be used in VECTOR

VECTOR[G -> {A rename v as w, B}] -- avoid name-clashes
VECTOR[frozen G -> ADDABLE] -- invariant generic param.

• Milyen lehetőségei vannak a constrained genericitynek Eiffelben?

Az Eiffelben a generikus típusoknak több (fix számú) típusparamétere lehet, és mindre tehetünk megszorításokat (az aktuális típusparamétereknek meg kell felelniük a formális típusparaméterekre tett megszorításoknak). Természetesen a generic típusdefiníción belül a típusparamétert olyan típusként használhatjuk, amely következik a rá adott megszorításokból. (Például olyan műveleteket használhatunk rá. . .)

Ha nem teszünk megszorítást, mint például a class G[T] esetben, az ekvivalens azzal, mintha azt írtuk volna, hogy class G[T->ANY]. Természetesen az ANY helyett más felső korlátot is adhatunk a típusparaméterre, pl: class G[T <- HASHABLE].

Még érdekesebb, amikor egy típusparaméterre több megszorítást adunk: ebben az esetben az aktuális paramétertípusnak az összes feltüntetett típus altípusa kell lenni. Például class G[T <- {HASHABLE, NUMERIC}].

Ha több típust is meghatározunk egy típusparaméter elvárt bázisaként, akkor ezeket szükséges lehet összehangolni, hogy konzisztensen lehessen használni a típusparamétert és a rá értelmezett műveleteket a generikus definícióban. Ez az összehangolás azt jelenti, hogy a különböző bázistípusokban lévő műveleteket átnevezhetjük, hogy a névütközést elkerüljük. Például class G[T <- {B rename f as g end, C}]. Ezek az átnevezések nem jelentik azt, hogy az aktuális típusparaméterben ilyen névvel kell legyenek deklarálva ezek a műveletek.

A típusparaméterre megadható az is, hogy egyedeket abból a típusból milyen creation procedure-ök segítségével lehet létrehozni. Ha például a B osztálynak van egy make nevű, paraméter nélküli művelete, akkor a class G[T <- B create make end] azt jelenti, hogy aT-nek megfeleltetett típus csak a B leszármazottja lehet, és ebben a típusban aB-ból megörökölt (és esetleg átnevezett)make‘ művelet egy creation procedure. Lehet olyan megkötést is adni egy típusparaméterre, hogy az aktuális csak olyan attached típus lehessen, amely öninicializáló, azaz amelynek az ANY-ból megörökölt (és esetleg átnevezett) default_create egy creation procedure-je. Például class G[?T].

Lehet olyan megkötést tenni, hogy a típusparaméter invariáns: class G[frozen T]. Ellenkező esetben a típusparaméter kovariáns.

Az Eiffelben nincs F-bounded polymorphism, azaz egy T típusparaméterre adott megszorításban nem hivatkozhatunk a T-re.

• Strukturális altípusosság esetén milyen altípusrelációk kell, hogy teljesüljenek az A, A0, B, B0, C, C0 típusokra, hogy matematikailag helyes legyen, hogy az (A → B) → C altípusa az (A0 → B0) → C0 típusnak?

A paraméter kontravariáns, az eredmény kovariáns kell legyen, ezért (A0 → B0) <: (A → B) és C <: C0 kell, továbbá az elsőhöz kell, hogy A <: A0 és B0 <: B.

• Mik az altípusosság szabályai függvénytípusokra? (9 ea)

A függvények értelmezési tartománya bővíthető, az értékkészlete szűkíthető származtatásnál.

A leszármazott osztályban az örökölt függvény visszatérési értékét specializálhatjuk/szűkíthetjük. Ez a kovarináns visszatérési érték. (kovariáns: együtt változó)

Legyen A <: B, és legyen T -ben egy f függvény B visszatérési típussal. Az S <: T akkor is fennáll, ha S újradeklarálja f-et A visszatérési típussal.

pl.:

class T
feature f: B

class S inherit T redefine f end
feature f: A

Paraméter változásnál kontravariancia (= Specializáció során általánosabb lesz) (azonban ezt nem szokták támogatni a programozási nyelvek):

Legyen A <: B, és legyen T -ben egy r rutin A típusú paraméterrel. Az S <: T akkor is fennáll, ha S újradeklarálja r-et B paramétertípussal.

pl.:

class T
feature r(p:A)

class S inherit T redefine f end
feature r(p:B)

(C++-ban viszont nincs ilyen, ott elfedést, és nem felüldefiniálást eredményez ez. A fordító statikus idejű döntése hogy melyik verziót választja)

(Javaban túlterhelés történik ekkor, itt is a fordító választ)

(Eiffelben a szerződések kontravarianciája támogatott, de paramétereké nem. Mondjuk lényegében ugyanaz a kettő.)

$A <: A'$, $B' <: B$

—----------

$A' \to B' <: A \to B$

PRE(r) ⇒ PRE(r) ∨ require_elser' =: PRE(r')

POST(r') := POST(r) ∧ ensure_thenr' ⇒ POST(r)

ahol r' felüldefiniálja a megörökölt r-t

Az Eiffel egy „kovariáns nyelv'' (Megengedi a paramétertípusok kovarianciáját, Rutinok paramétereire és Generikus típusok paramétereire, A szerződésekre viszont kontravariáns előfeltétel van (kontravariáns megkötés a paraméterre) Ez matematikailag helytelen, LSP-nek ellentmond -> Polymorphic CAT-call))

• Mi a Leggyengébb előfeltétel (weakest precondition)?

Edsger Wybe Dijkstra nevéhez fűződik.

Jelülése: $lf(S, P)$ vagy $wp(S, P)$

Ezen tulajdonságú állapotból lefuttatva az S programot, a program garantáltan véget ér, és P tulajdonságú állapotba kerül.

$lf(S, P) = \{a \in \mathcal{D}^{\check{}}(p(S)) \mid p(S)(a) \subseteq P\}$

• Mit jelent a rescue klóz?

Egy Eiffel rutin végén állhat egy rescue klóz, mely akkor fut le, amikor a rutin törzsében kivétel lép fel. A rescue klóz célja, hogy a rutin úgy érhessen véget, hogy legalább az osztályinvariáns teljesüljön, ha már az utófeltételt nem sikerült elérni. (A retry a rescue klózban a rutin újrakezdését kezdeményezi.) A szerződést (utófeltételt) teljesíteni nem tudó, és ezért kivételt kiváltó törzset követve lefut a

rescue, majd a fellépett kivételt propagálja a hívó felé, azaz a hívóban a hívás helyszínén fellép a szóban forgó kivétel. A hívó a szerződést teljesíteni nem tudó objektumot konzisztens állapotban látja (az osztályinvariáns teljesül rá).

Az r rutin retryr retry-klózának szerződése tehát:

{ true } retry r { inv C }

ahol inv C az r osztályának, azaz a C-nek az egyesített invariánsa (a 0bázistípusokban és a C-ben deklarált invariánsok konjunkciója).

Amennyiben egy rutin nem tartalmaz explicit retry-klózt, az ANY-ből megörökölt (és esetleg átnevezett) default_retry eljárás lesz implicit módon a rutin retry-klóza, melynek ANY-beli implementációja üres (SKIP utasítás). Egy lehetőség a retry-klózra egy creation-procedure végrehajtása (például a default_retry és a default_create ugyanolyan módon történő implementálása.)